

HW

jerrycyhu

14. April 2024

1 Problem Statement

In this project, we are doing the task of developing a Transformer Model for translating German sentences into English, focusing on the real use of a Transformer model, especially how to encode the token, how to decode it, how to do masking, and how the batch size will impact the performance of the model and training speed. Our primary dataset, the Multi30k dataset, contains pairs of German sentences and their English translations, which provides a foundation for both training and evaluating our model.

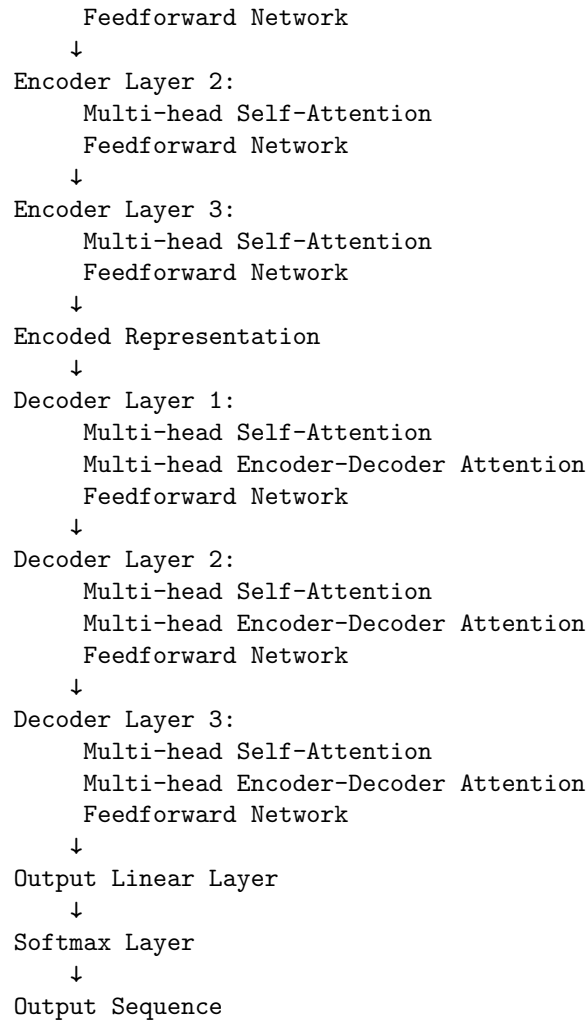
The process involves several key steps: preprocessing the text data to convert sentences into a machine-readable format using tokenization, training the Transformer model to understand and translate the language patterns, and finally, testing the model's ability to generate accurate English translations from new German sentences.

The effectiveness of the translation will be quantitatively assessed using the BLEU score. This project not only aims to achieve a high level of accuracy in translation but also serves as a practical application of advanced Transformer architectures in solving real-world problems in language understanding and translation.

2 Model description

2.1 Model Simple Illustration

```
Input Sequence
  ↓
Tokenization
  ↓
Token Embedding > Positional Encoding
  ↓
Encoder Layer 1:
  Multi-head Self-Attention
```



2.2 Explain

2.2.1 Positional Encoding

My Transformer uses positional encodings to inject information about the relative or absolute position of the tokens in the sequence. The positional encodings have the same dimension as the embeddings and are added to the input embeddings at the bottoms of the encoder and decoder stacks. This is crucial because the model itself doesn't have any recurrence or convolution; thus, without positional encodings, the model would see the sentence merely as a bag of words instead of a sequence of words. My implementation generates positional encodings based on sine and cosine functions of different frequencies.

2.2.2 Token Embedding

The token embedding module converts input tokens (from the vocabulary indices) into embeddings. In My model, tokens are first converted into embeddings which are then scaled up by the square root of the embedding size to help stabilize the learning process. This is followed by adding positional encodings.

2.2.3 Transformer Block

The core of the Transformer model consists of a stack of identical layers, each layer having two sub-layers in the encoder: a multi-head self-attention mechanism, and a simple, position-wise fully connected feed-forward network. The decoder, in contrast, inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack.

Encoder and Decoder Layers: Both consist of multiple layers (specified by 'num_encoder_layers' and 'num_decoder_layers') that include multi-head attention mechanisms and feed-forward networks. In My model, these numbers are both set to 3.

Multi-head Attention: Allows the model to jointly attend to information from different representation subspaces at different positions. With multiple attention heads, the model has the ability to focus on different parts of the input sequence simultaneously, improving the ability of the model to handle complex input patterns.

Feed-Forward Networks: These are simple fully connected neural networks applied to each position separately and identically. This means the same neural network is applied independently to each position.

2.2.4 Output Linear and Softmax

The output of the transformer decoder is passed through a linear layer followed by a softmax function to predict the probability distribution of the next token.

2.2.5 Key Functionalities

Masking: To prevent positions from attending to subsequent positions, the self-attention sub-layer in the decoder uses look-ahead masking. This masking, along with padding masks, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

Forward Pass: During the forward pass, embeddings are first created and combined with positional encodings. The transformed input then passes through the Transformer layers, where attention mechanisms and feed-forward networks process the sequence. Finally, a generator transforms the output of the Transformer to a size that matches the vocabulary, and a softmax is applied to predict the probability distribution of each token.

3 Experimental setup

3.1 Batch Size

The choice of batch size can significantly impact the training dynamics in terms of both memory usage and convergence behavior. For this project, I set the batch size to 16. This size strikes a balance between the efficiency of converging and the ability to generalize across different training examples. Smaller batch sizes often lead to faster convergence but can be noisy, while larger batches provide a more stable gradient but require more memory and may slow down the training process.

3.2 Gradient Accumulation

However, at first, I don't know what a good batch size is for my model, so I need to experiment with it. The problem I encountered when I was experimenting with the batch size was that my memory assigned to the training model would explode if I set the batch size over 1024. Gradient Accumulation helps to better test the batch-size without the need to worry about memory loads.

In my model training, gradient accumulation was used to effectively increase the batch size without increasing the actual batch size in memory. This method involves accumulating the gradients over several mini-batches and only updating the model weights after a specified number of iterations.

This technique allows the model to be trained with a larger effective batch size without requiring additional GPU memory, as only a fraction of the effective batch is loaded and processed at a time. It provides flexibility in managing the trade-off between training speed and hardware limitations.

Also by accumulating gradients, the model updates are based on a more comprehensive set of data, potentially leading to more stable and reliable gradient estimates, which can be particularly beneficial in noisy datasets or with complex models.

3.3 Weight Initialization

I used the Xavier uniform initialization for all the weights in the Transformer model. This initializer adjusts the scale of the weights according to the number of input and output neurons and is particularly suited for layers followed by sigmoidal activations like tanh:

```
for p in model.parameters():
    if p.dim() > 1:
        nn.init.xavier_uniform_(p)
```

3.4 Learning Rate and Optimizer

The learning rate controls how much we adjust the weights of our network with respect to the loss gradient. For this project, I used the Adam optimizer with a learning rate of 0.0001:

```
optimizer = torch.optim.Adam(model.parameters(),  
                               lr=0.0001, betas=(0.9, 0.98), eps=1e-9)
```

This configuration helps in dealing with the sparse gradients and varying data distributions that are typical in language tasks.

3.5 Training and Validation Setup

The training process involved iterating over the dataset in batches, where each batch consisted of a pair of source and target sequences. Gradient accumulation was also employed to stabilize the updates. This technique allows for larger effective batch sizes without increasing memory consumption, by accumulating the gradients over several mini-batches and updating the model weights less frequently.

Validation was performed at the end of each training epoch to monitor the model's performance on unseen data and prevent overfitting. The model's parameters that resulted in the lowest validation loss were saved, and early stopping was used based on the validation results to terminate training when the validation loss began to increase, indicating overfitting.

3.6 Regularization Techniques

To further combat overfitting, dropout was applied within the Transformer layers. A dropout rate of 0.1 was used to randomly omit a subset of features at each training step.

4 Experimental results

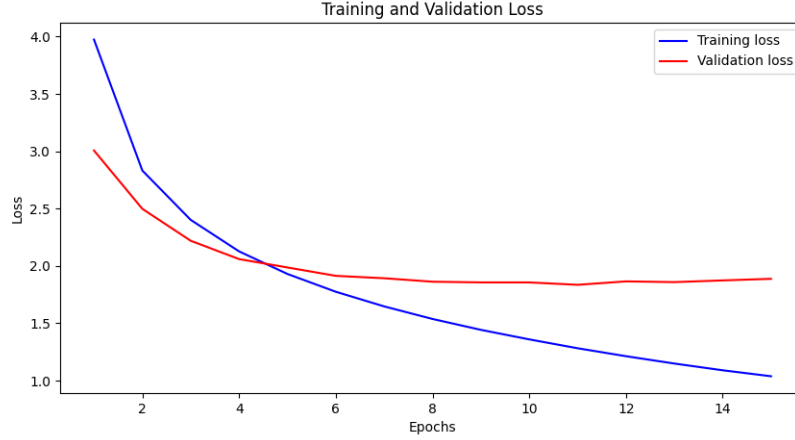


Fig.1: Training and Validation Loss. An intriguing observation from the initial stages of the experiment is that the average training loss was initially higher than the validation loss. Over time, however, the training loss began to exceed that of the validation.

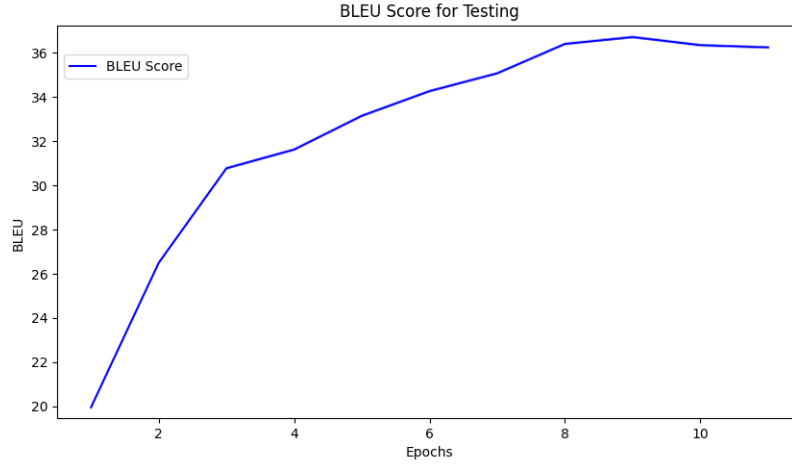


Fig.2: BLEU Score for Testing. Towards the final stages of the experiment, specifically in rounds nine and ten, there was a slight decline in the BLEU score. This decrease is linked to the model’s stopping strategy, which halts training if signs of overfitting are detected in two consecutive rounds—characterized by a decrease in training loss accompanied by an increase in validation loss. This approach aims to prevent the model from learning noise and irrelevant patterns from the training data, which can degrade performance on new, unseen data.

5 Analysis

An intriguing observation from the initial stages of the experiment is that the average training loss was initially higher than the validation loss. Over time, however, the training loss began to exceed that of the validation. Upon investigating this phenomenon, it became evident that this pattern is attributable to the composition of the datasets. The training dataset is considerably larger and includes a variety of complex cases, whereas the validation set is smaller and predominantly comprises simpler cases. Initially, when the model has not yet learned to generalize the knowledge it has acquired, the validation loss appears artificially lower than the training loss.

Moreover, Further analysis revealed a notable dependency on batch size with respect to model generalization. Specifically, smaller batch sizes tend to enhance the model's ability to generalize, rather than merely fitting to the training data. This is a fascinating comparison, demonstrating that even when models trained with different batch sizes achieve similar training losses, the model trained with smaller batches consistently outperforms the one trained with larger batches in both validation and testing phases.

This effect can be attributed to the increased noise introduced by smaller batches. Training with smaller batches, as opposed to using larger batches or the entire dataset at once, incorporates more variability during the learning process. This noise acts as a form of regularization, helping to prevent the model from overfitting to the training data and promoting better generalization to new data.

6 Conclusion

In this report, I detailed the development and evaluation of a Transformer model designed for a language translation task from German to English using the Multi30k dataset. The core activities included setting up the model architecture, preparing the data, conducting training sessions, and evaluating the model using BLEU scores to assess translation quality.

Building on the learnings from this assignment, several things can be further explored to enhance model performance and utility:

Experimenting with techniques like attention dropout or label smoothing to improve generalization and manage overfitting more effectively.

Utilizing automated tools like Bayesian Optimization to find the optimal set of hyperparameters, which could potentially enhance model performance without extensive manual testing.

Based on my halt strategy, making the model remember at least two rounds

before its updates would be better implemented. When the training stopped, it will be able to revert back to the third last updates, which is the most optimized version of the model.